

Bottleneck-Aware Schedule-First Tuning for Triton Kernels Under Matched Budget

Pedram

Giovanni Galea Curmi

April 2026

Abstract

Triton kernels expose many schedule choices, but trying every valid configuration is often too expensive to be a realistic baseline for day-to-day tuning. This paper asks a narrower question: given a fixed measurement budget, can a schedule-first tuner use cheap compile-adjacent signals and a small amount of profiling to spend that budget better?

We build KernelTuner, a matched-budget tuning framework that records which candidates are admissible, which cheap signals they expose, what limited profiling says, and how selector revisions change over time. We evaluate it on a homogeneous RTX A6000 pool using representative and aligned GEMM workloads as the main case study and regime-split LayerNorm workloads as a secondary case.

The results are mixed but clear. Cheap signals help with pruning, but they are not reliable enough to rank representative GEMM candidates on their own. Aligned GEMM makes the selector look better than it is, so it is useful as context but not as the main truth source. LayerNorm does not become a second positive profiling story: the small-batch regime is weak and noisy, while the large-batch regime regresses under the same budget.

The project also records one bounded negative result and one bounded positive result. A transfer-safe corrective pass on an expanded `split_k` space fails, which supports dropping `split_k` and `rows_per_program` from the final paper-facing surfaces. After that restriction, a conservative non-`split_k` mainline selector reaches 1.0286 geometric-mean speedup versus default on representative GEMM, compared with 0.8101 for the compile-ranked parent under the same budget. The main conclusion is narrow: lightweight signal-guided tuning can help, but only when workload realism, evidence discipline, and selector scope are kept explicit.

1 Introduction

GPU kernel performance depends not only on what computation is performed, but also on how that computation is scheduled on the hardware. That basic idea appears throughout prior work on schedule search and autoscheduling, from Halide through TVM, Ansor, and

later tensor-program optimization systems [Ragan-Kelley et al., 2012, Mullapudi et al., 2016, Adams et al., 2019, Chen et al., 2018a, Zheng et al., 2020, Chen et al., 2018b, Shao et al., 2022, Zheng et al., 2022]. Recent work also pushes farther toward full-kernel generation and agentic optimization [Dai et al., 2026]. This project studies a narrower problem.

Triton is a good setting for that narrower study. It exposes a compact set of kernel meta-parameters such as tile sizes, warp counts, grouping choices, and pipeline depth [Tillet et al., 2019, OpenAI, 2021, Triton Contributors, 2026a,b]. These choices are small enough to study directly, but large enough to matter for performance. In practice, a kernel author often falls back to a default schedule, a small random search, or a more expensive autotuning path through Triton or PyTorch tooling [PyTorch Contributors, 2026a,b]. That leaves a practical question: can a small fixed budget be spent more intelligently?

This paper studies that question as an empirical systems problem rather than as a black-box search benchmark. We use *matched budget* to mean that the compared selectors get the same measurement allowance. The goal is not to show that one selector can win after spending more profiling or more trials. The goal is to ask whether a bottleneck-aware, schedule-first selector can make better use of the same limited budget than a default choice or an equally budgeted naive baseline.

That framing places the project closer to external autotuning systems such as OpenTuner, Kernel Tuner, CLTune, and KTT than to compiler-scale schedule synthesis or full-kernel generation [Ansel et al., 2014, van Werkhoven, 2019, Nugteren and Corporaal, 2015, Petrov et al., 2023]. The selector matters, but so do the workload design and the evidence rules. Negative results therefore matter as much as positive ones. A transfer failure is not noise to hide if it changes what should stay on the final paper-facing surface.

The final project story is narrower than the initial exploration, but it is stronger for that reason. The evidence shows that cheap signals help more with pruning than with ranking, that aligned workloads flatter the selector, and that profiling is not uniformly helpful across kernels or regimes. The revised-selector story is also mixed: an idea that looked promising on a smaller space did not

carry over cleanly once `split_k` expanded the search surface. After that failure, the paper freezes a conservative non-`split_k` mainline and asks a simpler question. On that final surface, a guarded selector does recover the strongest representative GEMM result in the project. That mix of bounded wins and bounded failures is the main contribution.

We make four contributions:

- We present KernelTuner, a schedule-first matched-budget tuning framework for Triton kernels that cleanly separates reportable and diagnostic evidence.
- We provide a workload-aware empirical study across representative GEMM, aligned GEMM, and regime-split LayerNorm on a homogeneous RTX A6000 pool.
- We show how transfer failures, diagnostic studies, and keep/drop decisions strengthen the research story rather than weakening it, including bounded negative results for `split_k` and `rows_per_program`.
- We demonstrate that a final conservative mainline selector can materially improve representative GEMM under the same budget without reopening the research program.

The rest of the paper is organized as follows. Section 2 frames the tuning problem and workload roles. Section 3 positions the work relative to prior autotuning and profiling-guided tuning systems. Section 4 describes KernelTuner and the final paper-facing tuning surfaces. Section 5 describes the experimental protocol and evidence discipline. Section 6 presents the main empirical findings from cheap-signal limits through the final mainline result. Section 7 discusses what the project does and does not establish.

2 Background and Problem Setting

This paper treats Triton tuning as a schedule-selection problem in the spirit of prior autoscheduling work, but on a deliberately narrower surface. Halide and TVM-style systems search over richer schedule languages and broader compiler representations [Ragan-Kelley et al., 2012, Mullapudi et al., 2016, Adams et al., 2019, Chen et al., 2018a, Zheng et al., 2020, 2022]. Triton, by contrast, exposes a compact set of meta-parameters directly at the kernel level [Tillet et al., 2019, OpenAI, 2021]. That makes the space rich enough to be meaningful and small enough to study mechanistically. For GEMM, the critical knobs include tile geometry, reduction depth, grouped launch order, warp count, and pipelining depth. For LayerNorm, the active knobs are narrower, but work granularity and launch scale still matter. This paper does not study arbitrary algorithmic variants. It studies whether a bounded selector can reason about these schedule choices using lightweight evidence rather than broad brute-force search.

Item	Final choice
Research question	Can cheap compile-adjacent signals and bounded profiling guide Triton schedule selection better than default or naive matched-budget baselines?
Primary kernel	Representative GEMM
Secondary kernel	LayerNorm, split into <code>small_batch</code> and <code>large_batch</code> regimes
Environment	Homogeneous RTX A6000 pool across <code>gpunode2/gpunode3</code>
Headline result	Bounded positive non- <code>split_k</code> mainline result via <code>v5_mainline_profiled</code>
Closeout rule	No further selector-family expansion is justified in the paper backbone

Table 1: Study scope and final research framing.

Surface	Active knobs	Signals
GEMM final	<code>block_m</code> , <code>block_n</code> , <code>block_k</code> , <code>group_size_m</code> , <code>num_warps</code> , <code>num_stages</code>	Tier 0 compile signals + reportable <code>compute_lite</code>
LayerNorm final	<code>block_size</code> , <code>num_warps</code> , <code>num_stages</code>	Tier 0 compile signals + bounded LayerNorm profiling
Retired diagnostic knobs	<code>split_k</code> , <code>rows_per_program</code>	Diagnostic-only keep/drop evidence

Table 2: Final paper-facing knob surfaces and signal tiers.

Our interpretation vocabulary is bottleneck-driven. Cheap compile-adjacent signals such as register count, shared-memory footprint, and occupancy estimate help detect obviously poor candidates, but they are only indirect proxies for runtime behavior. Limited Tier 1 profiling adds a small amount of runtime evidence, but only on calibration subsets and only when counter availability is good enough to remain reportable. Everything else stays diagnostic-only. This distinction keeps the paper honest: the selector should not quietly consume evidence that the results section later treats as too fragile to trust.

The workload program is equally important. Representative GEMM is the paper’s truth source because it includes irregular aspect ratios and masked edges that can expose ranking failures. Aligned GEMM is deliberately retained as a supporting context workload because it can make the selector look better than it really is. LayerNorm plays a different role: it is a secondary validation kernel used to test whether bounded profiling helps more on a workload with stronger memory behavior. The final evidence shows that this story is regime-split rather than uniformly positive.

The project’s hypotheses were intentionally staged. Early rounds asked whether cheap signals prune better than they rank and whether aligned workloads flatter the selector. Middle rounds asked whether bounded profil-

Kernel	Workload classes	Role
Representative GEMM	<code>square_compute</code> , <code>m_dominant</code> , <code>n_dominant</code> , <code>edge_nondivisible</code>	Primary truth source for ranking quality and the final headline
Aligned GEMM	<code>aligned_square</code> , <code>aligned_rectangular</code>	Context workload used to check whether regular shapes flatter the selector
LayerNorm	<code>small_batch</code> , <code>large_batch</code>	Secondary regime-split case for interpreting profiling value

Table 3: Final workload matrix used by the draft.

ing or frontier-aware revisions transfer. The final round did not introduce a new hypothesis. Instead, it froze a conservative non-`split_k` surface and asked whether one last guarded selector family could produce a credible final mainline result. That separation matters for interpretation: the paper’s positive headline comes from a final consolidation pass, not from rewriting the earlier transfer failure into a success story.

This framing also distinguishes the project from recent agentic GPU optimization work. Systems such as CUDA Agent target full CUDA kernel generation or broader search over implementation programs [Dai et al., 2026]. Our goal is narrower: keep the Triton implementation fixed, keep the budget explicit, and make it possible to explain both wins and failures with recorded bottleneck evidence.

3 Related Work

Our work sits at the intersection of autoscheduling, tensor-compiler autotuning, practical GPU kernel autotuners, and recent agentic GPU optimization. The closest category depends on which level of abstraction one emphasizes, so the paper should be explicit about what it does *not* claim as much as what it does.

Autoscheduling and schedule search. The broadest conceptual lineage comes from Halide, which made schedule choice an explicit optimization object rather than an implementation detail intertwined with correctness [Ragan-Kelley et al., 2012]. Subsequent Halide autoschedulers showed that schedule spaces could be searched automatically, including with learned guidance and structured exploration [Mullapudi et al., 2016, Adams et al., 2019]. This line of work establishes the central premise behind our study: schedule selection can dominate performance even when the high-level computation is fixed. Our setting is much narrower, however. We do not search over a rich transformation language. We study a bounded Triton meta-parameter space whose knobs already have

direct operational meaning for kernel authors.

Tensor-compiler autotuning and learned cost models. TVM, AutoTVM, Ansor, MetaSchedule, and TensorIR push schedule search into richer compiler and IR-level optimization settings [Chen et al., 2018a,b, Zheng et al., 2020, Shao et al., 2022, Zheng et al., 2022]. These systems are scientifically adjacent because they also combine structured search with predictive guidance, but their objective is broader than ours. They ask how to optimize tensor programs or compiler schedules in general. We ask whether a much smaller Triton configuration space can be searched effectively with cheap compile-adjacent signals and limited profiling under a matched-budget rule. In that sense, our contribution is not a stronger search algorithm. It is a cleaner empirical study of what limited evidence can and cannot buy on a practical fixed-kernel surface.

Triton-native tuning. Triton is the immediate technical context for this paper. Its language and compiler make schedule-oriented GPU programming practical while still exposing performance-critical parameters such as tile sizes, warp counts, and pipeline stages [Tillet et al., 2019, OpenAI, 2021]. Triton also includes built-in tuning hooks such as `triton.autotune` and `triton.heuristics` [Triton Contributors, 2026a,b]. More broadly, PyTorch increasingly treats Triton as part of the compilation and autotuning stack, including autotune-oriented `torch.compile` modes and user-defined Triton-kernel workflows [PyTorch Contributors, 2026a,b]. This ecosystem is important because it shows that bounded tuning is not merely academic. The open question is whether that tuning can be made cheaper, more interpretable, and more reproducible than straightforward benchmark-driven search. That question is where our work contributes.

External kernel autotuners. The practical architecture of KernelTuner is closest to external autotuning frameworks such as OpenTuner, Kernel Tuner, CLTune, and the Kernel Tuning Toolkit [Ansel et al., 2014, van Werkhoven, 2019, Nugteren and Corporaal, 2015, Petrov et al., 2023]. These systems treat kernels as parameterized programs, generate candidate configurations, validate correctness, benchmark performance, and compare search strategies. Our system follows that tradition more closely than it follows compiler-level autoscheduling. The main difference is scope and evidence discipline: KernelTuner specializes to Triton, enforces matched-budget comparisons, separates reportable profiling from diagnostic profiling, and treats failure analysis and keep/drop decisions as first-class outputs rather than side effects of a search benchmark.

Recent agentic GPU optimization. Recent work such as CUDA Agent treats GPU optimization as a

broader agentic synthesis problem over full CUDA kernels [Dai et al., 2026]. This is valuable context because it shows how quickly the automation frontier has moved. It also sharpens our positioning. We are not claiming to compete with systems that rewrite or synthesize full kernels. Our study keeps the Triton kernel fixed and asks a stricter question: given a bounded schedule space and a fixed budget, can lightweight signals guide selection better than default or naive baselines, and can the resulting failures be explained credibly? That narrower scope is precisely what lets the paper make strong matched-budget and negative-result claims.

Practical ecosystem motivation. The surrounding Triton ecosystem suggests that tuning remains an active engineering problem even when native autotuning support exists. Official tutorials present tuning as standard practice for performance-critical kernels, and tools such as `triton-dejavu` target the overhead and reuse problems around repeated autotuning [IBM, 2025]. We treat this ecosystem evidence as motivation rather than as the paper’s scientific foundation. It explains why a lightweight, reproducible, artifact-rich tuning framework is still worth studying even in a setting with existing autotune support.

Taken together, the literature suggests a clear niche for this paper. Rich schedule spaces, learned cost models, and full-kernel generation systems all matter, but they are not the only meaningful research targets. There is still value in a narrower Triton configuration study that prioritizes scope discipline, matched-budget fairness, and explanation quality. That is the niche KernelTuner is intended to occupy.

4 KernelTuner Design

KernelTuner is designed around one principle: the selector should consume only the kinds of evidence that the paper is willing to defend. The implementation therefore emphasizes admissibility, matched-budget comparisons, and explicit separation between reportable and diagnostic signals.

Candidate generation and admissibility. The generator enumerates the declared schedule space, validates kernel-specific configurations, and treats admissibility limits explicitly rather than silently truncating the raw Cartesian space. This design became important once the GEMM space expanded: if valid candidates are hidden by pre-validation truncation, the experiment stops measuring selectors and starts measuring config ordering artifacts.

Selector ladder. The implementation uses a fixed selector ladder: `default_config`, naive random and grid baselines, cheap-signal pruning and ranking, bounded profiling, and revised selectors under the same budget. The

Candidate generation → enumerate declared schedule space, validate configs, and keep only admissible kernels. Tier 0 signals → correctness, compile success, register count, shared memory, occupancy estimate. Frontier construction → cheap-signal pruning plus compile ranking or guarded revised frontier. Optional Tier 1 profiling → bounded calibration-only profiling with reportable counter sets. Selection → choose one configuration under matched budget. Held-out evaluation → benchmark on workload classes, compare against default and naive baselines, and write study artifacts. Evidence lock → promote only stable repo-local campaign, study, and bundle outputs into claims.

Figure 1: Conceptual overview of the KernelTuner pipeline used throughout the project. The implementation is intentionally structured around evidence discipline: every selector decision and every promoted claim must trace back to a stable artifact.

ladder is not just an engineering convenience. It structures the hypotheses and forces every claimed improvement to be compared against a matched-budget parent.

Signal tiers. Tier 0 signals are broad and cheap. Tier 1 signals are bounded and reportable only when availability and attribution are defensible. Diagnostic sets are allowed for explanation but not for superiority claims. This is why the paper can use schedule diagnostics to explain the failure of `split_k` without treating those counters as part of the main tuning contract.

Final paper-facing surfaces. The final mainline surfaces are intentionally narrower than the exploratory surfaces used along the way.

- **GEMM final surface.** The paper-facing GEMM surface retains `block_m`, `block_n`, `block_k`, `group_size_m`, `num_warps`, and `num_stages`. It excludes `split_k`.
- **LayerNorm final surface.** The paper-facing LayerNorm surface retains `block_size`, `num_warps`, and `num_stages`. It fixes `rows_per_program=1`.

Retired knobs remain implemented, but only as archived diagnostic surfaces. This keeps the paper faithful to the implementation while still drawing a firm boundary around what counts as the final mainline result.

Final guarded selector family. The last code-level change in the project was deliberately conservative. Instead of adding another broad revision family, the final pass introduced a guarded non-`split_k` frontier union that preserved the strongest compile-ranked families while recovering balanced candidates that earlier revisions had missed or overcorrected past. The final abla-

tion shows that once this conservative frontier is in place, most of the gain is already present before profiling, and profiling adds only a small extra increment.

5 Experimental Methodology

The methodology is designed to answer one narrow question fairly: under the same measurement budget, does a bottleneck-aware selector choose better configurations than default settings or naive search baselines?

Environment. All promoted primary studies use a homogeneous RTX A6000 pool across `gpunode2` and `gpunode3`. The research documentation treats these nodes as one qualified environment, not as separate experimental conditions. This matters because the final paper uses repeated and cross-run evidence gathered across both nodes while still making only single-architecture claims.

Matched-budget protocol. Every comparative result in the paper is budget matched. Default, naive random search, compile-ranked selection, profiled selection, and revised selectors all operate under explicit run limits. Diagnostic studies are allowed to collect extra explanation, but those results are never promoted as if they were part of the main matched-budget comparison.

Repeatability and robustness. The project uses two complementary modes:

- **Repeatability:** three repeated runs with the same seed.
- **Robustness:** one run each at seeds 7, 19, and 43.

This separation matters in the final headline. The guarded `v5_mainline_profiled` result is clearly positive on mean performance, but it is still described as a bounded improvement because it does not clear the stricter positive-seed gate required for the strongest promotion tier.

Promotion rules. The paper only promotes evidence that survives three filters:

1. the relevant campaign and study completed successfully;
2. the source artifact is repo-local and canonical;
3. the claim wording is admitted by the final claim inventory.

This is why the final figure set reuses Phase 2 aligned-context evidence and Phase 3 schedule diagnostics even after the R6 run. The optional aligned refresh was skipped by gate, so the stronger earlier aligned comparison remains the paper-safe source.

Evidence discipline. The final promotion boundary is the bundle under `artifacts/analysis/final_paper_20260330/`. Paragraph-level results in the paper must trace to both a claim row in `docs/research/11_final_claim_inventory.md` and a canonical artifact path in the final bundle. Incomplete or superseded campaign roots, scratch-only experiment trees, and archived raw Phase 3 roots moved under `/tmp/.../phase3_raw` are intentionally excluded from promotion.

6 Results

6.1 Cheap Signals and Workload Realism

Unless stated otherwise, the numeric values in this section are geometric-mean speedup relative to the Triton default schedule. A value above 1.0 is an improvement, while a value below 1.0 is a regression.

The first stable result in the project is simple: cheap compile-adjacent signals help most when they rule out bad candidates early. They do much less well at deciding which of the remaining candidates is actually best on representative GEMM.

The clearest evidence comes from the Phase 2 representative GEMM study on the final non-`split_k` surface. Under the same budget, naive random search reaches a geometric-mean speedup of 1.0331 versus default, while the compile-ranked parent `prune_rank` reaches only 0.8265. Adding bounded profiling raises the result only modestly to 0.8444. The early revised selector falls much further, to 0.1458. The main point is not that cheap signals are useless. The point is that they are better at pruning obviously weak candidates than at ranking several plausible survivors.

That result is easier to see once the workload is chosen carefully. In this paper, representative GEMM is the truth source because it includes the irregular shapes that expose ranking mistakes. Figure 2 shows why the aligned workload is kept only as context. On aligned GEMM, the same strategies look better: `prune_rank` rises from 0.8265 to 0.8795, and `prune_rank_profiled` rises from 0.8444 to 0.8873. If the evaluation had stopped there, the selector would have looked stronger than it really is on the more realistic workload matrix.

6.2 LayerNorm as a Bounded Regime-Split Result

LayerNorm answers a different question. It does not become a second headline optimization win. Instead, it shows that bounded profiling has a mixed effect that depends on workload regime.

The regime split matters immediately. In the Phase 2 reportable studies, `small_batch` shows only a very small gain from profiling: `prune_rank=1.0100` and

ID	Status	Paper-safe reading	Main evidence
H1	Strong	Cheap compile-adjacent signals prune effectively but do not reliably rank representative GEMM on realistic schedule surfaces.	Phase 2 representative GEMM
H2	Mixed / bounded	LayerNorm is regime-split and secondary: small_batch is weak/noisy, large_batch favors compile-only ranking.	Phase 2 regime split + Phase 3 microstudy
H3	Contextual	Aligned GEMM is useful context, but it is not the truth source and overstates selector quality.	Phase 2 aligned vs representative comparison
H4	Mixed transfer story	Narrow-space frontier-aware revision worked, but later expanded-space evidence did not transfer cleanly.	h4_retry_g3 , Phase 3 ablation, R6 ablation
H5	Unsupported	The transfer-safe v4 corrective pass remained unsupported on the expanded split_k surface.	Phase 3 representative GEMM + schedule diagnostic
R6	Bounded improvement	The guarded non- split_k mainline selector yields the strongest representative GEMM result in the project.	Final representative GEMM + final ablation

Table 4: Final hypothesis and claim summary used by the manuscript.

Strategy	Rep. GEMM	Aligned GEMM
prune_rank	0.8265	0.8795
prune_rank_profiled	0.8444	0.8873
naive_random_search	1.0331	0.9951
Source. Phase 2 canonical aligned-context comparison from gemm_v2_baseline_mapping and gemm_v2_aligned_reference .		

Figure 2: Aligned GEMM remains useful as context, but it is not the truth source. All values are geometric-mean speedup versus the Triton default schedule. Compile-ranked strategies look better on aligned workloads than on the representative GEMM matrix.

prune_rank_profiled=1.0113. In **large_batch**, profiling moves in the wrong direction: **prune_rank**=1.0029 while **prune_rank_profiled**=0.9856. The later Phase 3 microstudy does not change that reading. It only strengthens the decision to remove **rows_per_program** from the paper-facing LayerNorm surface.

This is still a useful result. It shows that even when profiling provides extra information, that information does not automatically translate into a robust matched-budget win. It also shows why LayerNorm belongs in the paper as a bounded secondary story rather than as a second main result.

6.3 Transfer, Failure Analysis, and Keep/Drop Decisions

The revised-selector part of the project is easiest to understand as a transfer story. On a narrower search space, the frontier-aware revision looked promising. Once the search space expanded, that same idea stopped carrying over cleanly.

Strategy	Small batch	Large batch
prune_rank	1.0100	1.0029
prune_rank_profiled	1.0113	0.9856
prune_rank_revised	1.0080	0.9964
Source. Phase 2 regime-split LayerNorm studies. Phase 3 microstudies support the same bounded interpretation and retire rows_per_program .		

Figure 3: LayerNorm is a regime-split secondary result, not a major positive profiling story. All values are geometric-mean speedup versus the Triton default schedule. Small-batch profiling is weak and noisy, while large-batch profiling regresses relative to compile-only ranking.

The Phase 3 **split_k** studies are the clearest limit case. On the expanded representative GEMM surface, **prune_rank** reaches 1.0324 and naive random search reaches 1.0427, but **v4_transfer_safe_profiled** falls to 0.1609. The matching ablation makes the problem even clearer: frontier-only and profiled v4 are nearly identical at 0.1622 and 0.1623, both far below the parent at 0.9692. This is why the project does not present revised selectors as a simple success story.

That failure is useful because it closes two design decisions cleanly. First, **split_k** does not stay on the mainline GEMM surface. Second, **rows_per_program** does not stay on the mainline LayerNorm surface. The schedule diagnostic supports the GEMM decision directly: in the canonical diagnostic run, both the selected and the best-scored family converge to a balanced 128x128x32, **group_size_m**=4, **split_k**=1 schedule, while non-unit **split_k** values remain dominated alternatives.

Final mainline ablation	
Selector	Geomean speedup vs default
Parent <code>prune_rank</code>	0.8103
<code>v5_mainline_frontier</code>	1.0272
<code>v5_mainline_profiled</code>	1.0283

Phase 3 schedule diagnostic	
Diagnostic fact	Value
Selected family	$128 \times 128 \times 32$
<code>group_size_m</code>	4
<code>split_k</code>	1
Selected equals best scored	Yes
Score regret	0.0

Figure 4: Left: final non-`split_k` mainline ablation. Right: Phase 3 schedule diagnostic showing that the canonical selected family remains balanced and non-`split_k`, while non-unit `split_k` values stay diagnostically dominated rather than promoted.

Strategy	Geomean speedup vs default
<code>prune_rank</code>	0.8101
<code>prune_rank_profiled</code>	0.8092
<code>naive_random_search</code>	1.0011
<code>v5_mainline_profiled</code>	1.0286

Source. R6 final mainline study `gemm_final_baseline_mapping`. The final bundle records this as a bounded improvement because the stricter positive-seed gate did not clear.

Figure 5: Final representative GEMM headline. The guarded non-`split_k` mainline selector materially improves over the compile-ranked parent and approaches naive random search under the same matched budget.

6.4 Final Mainline Consolidation

The paper’s positive headline comes from the final non-`split_k` mainline pass. This step does not reopen the search program. It narrows the paper-facing surface and asks whether one conservative selector family can recover a credible final result under the same matched budget.

Figure 5 shows that it can. On representative GEMM, the guarded `v5_mainline_profiled` selector reaches 1.0286 geometric-mean speedup versus default. That is 0.2185 speedup points above the compile-ranked parent at 0.8101, and it also exceeds naive random search at 1.0011 under the same budget. The paired ablation in Figure 4 shows where that recovery comes from. Most of it comes from conservative frontier construction, while profiling adds only a very small extra increment once the frontier is already in better shape.

This final result should be read narrowly. It does not show that Triton autotuning is solved, and it does not erase the earlier transfer failure. What it does show is that a conservative selector on the final paper-facing surface can recover a real, bounded representative GEMM improvement after several mixed and negative rounds. That combination of recovery and restraint is the main result of the paper.

7 Discussion and Limitations

The project ends with a narrower story than it started with, but it also ends with a stronger one. The main lesson is that lightweight signals help most when they are used carefully. They are useful for pruning and for building a better frontier of candidates, but they are not enough to solve final ranking on their own.

Why compile signals prune better than they rank. Compile-adjacent signals can expose obvious problems such as excessive resource use or poor occupancy potential. What they do not capture well is the finer runtime tradeoff between several candidates that are all already valid. That is why the project repeatedly sees a gap between compile-ranked selection and the best representative GEMM family reachable under the same budget.

Why aligned workloads mislead. Aligned GEMM is still useful, but only in the right role. It shows how the selector behaves on regular shapes. It becomes misleading when treated as the main truth source, because regular workloads make the selector look better than it does on the irregular representative matrix. Keeping aligned GEMM in the paper is therefore useful precisely because it warns against overreading an easier evaluation setting.

Why LayerNorm stayed secondary. LayerNorm stayed secondary not because it was unimportant, but because the evidence never supported a strong matched-budget win. The regime split still taught something important. It showed that a pooled interpretation was too coarse, and it supported the decision to remove the extra LayerNorm knob from the final paper-facing surface.

Why transfer failed on the expanded `split_k` space. The negative Phase 3 result is part of the paper’s credibility, not a detour to hide. A selector family that looked promising on a narrower space did not survive the expanded `split_k` surface, and the diagnostic evidence shows that the added schedule family did not become a useful mainline choice. That bounded failure is exactly what justifies the keep/drop decisions and the later return to a conservative final surface.

Why the final result is intentionally conservative.

The final mainline selector is not a last-minute attempt to rescue every earlier result. It is a bounded corrective pass on the final non-`split_k` surface. That should stay explicit in the paper. The strength of the result comes from the combination of a real positive outcome and disciplined scope control, not from implying that the selector is broadly robust.

What this could mean for open-source tooling.

The work is narrow, but parts of it could still transfer into open-source Triton tooling. The clearest candidates are the cheap-signal pruning rules, the matched-budget evaluation protocol, and the evidence split between reportable and diagnostic profiling. These are easier to adopt than the full selector story because they do not require claiming that one learned or revised policy generalizes broadly. In other words, the paper supports extending existing tuning libraries with better pruning and better evaluation discipline more strongly than it supports shipping a universal Triton selector.

The paper also has clear non-claims:

- It is not a universal Triton autotuner.
- It does not make multi-architecture claims.
- It does not show that profiling always helps.
- It does not show that revised selectors always transfer across search spaces.

These limits are part of the contribution, not a weakness in how the paper is framed. The project is useful because it says clearly where the method helps, where it does not, and why the final paper-facing system keeps only the parts that survived the evidence lock.

8 Conclusion

This paper asked whether a bottleneck-aware, schedule-first selector could use cheap compile-adjacent signals and bounded profiling to tune Triton kernels better than default settings or equally budgeted naive baselines. The final answer is yes, but only in a constrained and carefully evidenced sense.

The project delivers one strong primary GEMM story: cheap signals prune better than they rank, aligned workloads overstate selector quality, transfer can fail on expanded spaces, and a guarded final non-`split_k` mainline selector recovers the strongest representative GEMM result in the project. It also delivers one bounded secondary LayerNorm story: profiling is regime-dependent, weak in `small_batch`, and negative in `large_batch` under the current matched-budget setup. Finally, it delivers a meaningful negative-result contribution: the transfer-safe `split_k` corrective pass is unsupported, and the resulting keep/drop decisions improve the credibility of the final system rather than weakening it.

Taken together, these results suggest that lightweight signal-guided tuning is most valuable when it is treated as a disciplined evidence problem rather than as a universal autotuning solution. That is the paper’s final contribution: not a claim that Triton tuning is solved, but a clear demonstration of where matched-budget bottleneck-aware tuning helps, where it fails, and how to tell the difference.

A Extended Methods and Round Structure

C Provenance and Promotion Rules

This appendix records the round structure that led to the final narrative. The main paper intentionally avoids narrating every round chronologically, but the development path matters for provenance and handoff.

The implementation changed along the way, but the paper-facing method remained disciplined:

- selector revisions were admitted only when motivated by observed failure modes;
- reportable and diagnostic profiling were kept separate;
- canonical artifact bundles were regenerated after major rounds;
- and final claims were locked only after the paper bundle and final claim inventory agreed on their sources.

B Extended Results and Exact Artifact Mapping

The final manuscript uses a narrow figure set, but the underlying evidence is broader. Table 6 records the research-backed source map used by the first draft. In this repo snapshot, the paper backbone is tracked through the research docs, the ADRs, and the dated analysis logs.

The most important numeric comparisons used directly in the draft are:

- **Phase 2 representative GEMM:** random 1.0331, prune_rank 0.8265, prune_rank_profiled 0.8444, revised 0.1458.
- **Phase 2 aligned GEMM:** random 0.9951, prune_rank 0.8795, prune_rank_profiled 0.8873, revised 0.1702.
- **Phase 2 LayerNorm small-batch:** prune_rank 1.0100, profiled 1.0113.
- **Phase 2 LayerNorm large-batch:** prune_rank 1.0029, profiled 0.9856.
- **Phase 3 representative GEMM with split_k:** parent 1.0324, random 1.0427, transfer-safe profiled 0.1609.
- **R6 final representative GEMM:** parent 0.8101, random 1.0011, guarded profiled 1.0286.

These comparisons are sufficient for the first draft because they support the final claim ledger directly. Additional per-class breakdowns, stability reports, and opportunity catalogs should remain appendix or supplement material unless a later revision needs them in the main paper.

The first draft relies on the final claim inventory together with the local research docs, ADRs, and dated synthesis logs as the promotion boundary.

Canonical promotion sources. All promoted claim and figure sources must be repo-local, completed, and canonical. In practice this means:

- final claim wording comes from docs/research/11_final_claim_inventory.md;
- figure and table sources are reconstructed in this checkout from docs/research/08_evidence_registry.md, docs/adr/, and the dated synthesis logs under docs/research/logs/;
- historical interpretation comes from the backbone research docs and dated synthesis logs.

Explicit exclusions. The draft must not promote evidence from:

- incomplete or superseded campaign roots,
- scratch-only experiment directories,
- raw Phase 3 roots moved under /tmp/.../phase3_raw,
- optional aligned-refresh or confirmation reruns that were intentionally skipped by gate.

Why the final claim set is narrower than the project history. The project explored more configurations, revisions, and diagnostic paths than appear in the final main text. That narrowing is intentional. The final paper is not meant to re-narrate every operational step. It is meant to present the strongest defensible story that survives the final evidence lock. The dated logs remain the right place to preserve the full chronology and iteration history for handoff and future follow-up work.

Round	Theme	Outcome	Role in paper
R0	measurement validity	Established profiler availability, execution integrity, and basic reportability on the homogeneous A6000 pool.	Method / setup support
R1	cheap-signal baseline mapping	Shown that cheap signals help prune but do not reliably rank representative GEMM.	Core support for H1
R2	limited profiling	Shown that profiling does not yield the intended cross-kernel LayerNorm advantage.	Core support for bounded H2
R3	opportunity-guided refinement	Produced a narrow frontier-aware positive result that later became the positive side of the transfer story.	Context for H4
R4	transfer and limits	Consolidated the strongest and weakest cases into a stable evidence layer.	Background to later rounds
R5	bounded corrective transfer pass	Shown that the transfer-safe <code>split_k</code> revision family remained unsupported and justified keep/drop decisions.	Core support for H5 and diagnostic sections
R6	final mainline lock	Recovered the final non- <code>split_k</code> representative GEMM headline under a conservative guarded frontier.	Final main text headline

Table 5: Round-by-round research structure. The main paper is organized by story rather than by round, but the rounds explain why the final claim set is narrower and stronger than the initial exploration.

Artifact role	Canonical source	Used by
Final representative headline	<code>gemm_final_baseline_mapping</code> with supporting interpretation in <code>docs/research/08_evidence_registry.md</code> , <code>docs/research/11_final_claim_inventory.md</code> , and <code>docs/research/logs/2026-03-30_r6_final_synthesis_and_evidence_lock.md</code>	Figure 2, Table 4
Final mainline ablation Aligned context	<code>gemm_final_selector_ablation</code> plus the same R6 synthesis sources above <code>gemm_v2_aligned_reference</code> plus <code>gemm_v2_baseline_mapping</code> , interpreted through <code>docs/research/08_evidence_registry.md</code>	Figure 5, Table 4 Figure 3
LayerNorm regime split	<code>layernorm_v2_small_regime</code> plus <code>layernorm_v2_large_regime</code> , with bounded follow-up from the Phase 3 microstudy rows in <code>docs/research/08_evidence_registry.md</code>	Figure 4
Phase 3 transfer failure	<code>gemm_v3_baseline_mapping</code> and <code>gemm_v3_selector_ablation</code> , anchored by <code>docs/research/08_evidence_registry.md</code> and <code>docs/research/11_final_claim_inventory.md</code>	H5 text, Table 4
Chosen-vs-best diagnostic	<code>gemm_v3_schedule_diag</code> with the keep/drop decision documented in <code>docs/research/08_evidence_registry.md</code>	Figure 5B
Paper backbone	<code>docs/research/</code> , <code>docs/adr/</code> , and dated analysis logs under <code>docs/research/logs/</code>	Whole draft

Table 6: Canonical artifact map for the first draft.

References

- Andrew Adams, Karren Ma, Leia Anderson, Riyadh Baghdadi, Tianzhe Li, Michaël Gharbi, Benoit Steiner, Steven Johnson, Kayvon Fatahalian, Frédo Durand, et al. Learning to optimize halide with tree search and random programs. In *ACM Transactions on Graphics*, 2019.
- Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O’Reilly, and Saman Amarasinghe. Opentuner: An extensible framework for program autotuning. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, 2014.
- Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Yuwei Wang, et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation*, 2018a.
- Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Yuwei Wang, Haichen Shen, et al. Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems*, 2018b.
- Weinan Dai, Hanlin Wu, Qiying Yu, Huan-ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, Xin Liu, and Hao Zhou. CUDA agent: Large-scale agentic RL for high-performance CUDA kernel generation. *arXiv preprint arXiv:2602.24286*, 2026.
- IBM. triton-dejavu. <https://github.com/IBM/triton-dejavu>, 2025. Accessed March 2026.
- Ravi Teja Mullapudi, Andrew Adams, Dillon Sharlet, Jonathan Ragan-Kelley, and Kayvon Fatahalian. Automatically scheduling halide image processing pipelines. In *ACM SIGGRAPH 2016 Conference Papers*, 2016.
- Cedric Nugteren and Henk Corporaal. CLTune: A generic auto-tuner for OpenCL kernels. In *2015 8th International Workshop on Many-Core Computing Systems*, 2015.
- OpenAI. Introducing triton. <https://openai.com/index/triton/>, 2021. Accessed March 2026.
- Tomáš Petrov et al. Kernel tuning toolkit. *SoftwareX*, 22: 101391, 2023.
- PyTorch Contributors. `torch.compile` documentation. <https://docs.pytorch.org/docs/stable/generated/torch.compile.html>, 2026a. Accessed March 2026.
- PyTorch Contributors. Using user-defined triton kernels with `torch.compile`. https://docs.pytorch.org/tutorials/recipes/torch_compile_user_defined_triton_kernel_tutorial.html, 2026b. Accessed March 2026.
- Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Decoupling algorithms from schedules for easy optimization of image processing pipelines. *ACM Transactions on Graphics*, 31(4), 2012.
- Yidi Shao, Minmin Sun, et al. Tensor program optimization with probabilistic programs. *Advances in Neural Information Processing Systems*, 2022.
- Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.
- Triton Contributors. Triton `triton.autotune` documentation. <https://triton-lang.org/main/python-api/generated/triton.autotune.html>, 2026a. Accessed March 2026.
- Triton Contributors. Triton `triton.heuristics` documentation. <https://triton-lang.org/main/python-api/generated/triton.heuristics.html>, 2026b. Accessed March 2026.
- Ben van Werkhoven. Kernel tuner: A search-optimizing gpu code auto-tuner. *Future Generation Computer Systems*, 90: 347–358, 2019.

Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, et al. An-sor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation*, 2020.

Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, et al. Tensorir: An abstraction for automatic tensorized program optimization. *arXiv preprint arXiv:2207.04296*, 2022.